TEMCO LEARN

T3000 API & Protocol

Data Structures Reference · built Nov 13, 2026

Data Structures Reference

Data structures define the format and fields of each BACnet point type. Understanding these structures helps you work with device data effectively.

Key Points:

- All structures are C++ structs defined in `ud_str.h`
- Fields are packed for network transmission
- Values often use fixed-point representation (e.g., 720 = 72.0°F)

Overview

T3000 uses C++ structures to represent BACnet point data. These structures mirror the device's memory layout for efficient reading/writing.

Header File: `T3000-Source/T3000/CM5/ud_str.h`

Common Pattern

All point structures follow a similar pattern:

C++

```
typedef struct {
    char description[DESCRIPTION_LENGTH];  // Full description
    char label[LABEL_LENGTH];              // Short label
    // ... type-specific fields ...
    uint8_t auto_manual;                   // 0=Auto, 1=Manual
    uint16_t value;                        // Current value
    uint8_t range;                         // Unit/range
} Str_xxx_point;
```

Input Structure

Str_in_point

Size: 46 bytes **Location:** ud_str.h line ~421 **Max Count:** 64

C++

```
typedef struct {
    union {
        char description[STR_IN_DESCRIPTION_LENGTH+1];  // 21 bytes
    };
    union {
        char label[STR_IN_LABEL_LENGTH+1];  // 9 bytes
    };
    uint8_t auto_manual;        // 0=Auto, 1=Manual
    unsigned short value;       // Current value (scaled)
    unsigned char filter;       // Filter strength (0-5)
    unsigned char control;      // Control type
    unsigned char digital_analog;  // 0=Analog, 1=Digital
    unsigned char range;        // Unit/range index
    signed char calibration_sign;  // Calibration direction
    unsigned short calibration_h;  // High calibration
    unsigned short calibration_l;  // Low calibration
    unsigned char decom;        // 0=Normal, 1=Open, 2=Shorted
    unsigned char sub_id;
    unsigned char sub_product;
    unsigned char unused[4];
} Str_in_point;
```

Field Details

Field	Type	Range	Description
`description`	string	20 chars	Full name
`label`	string	8 chars	Short name
`value`	uint16	0-65535	Scaled value (÷10 for actual)
`auto_manual`	uint8	0-1	0=Auto, 1=Manual
`filter`	uint8	0-5	Filter level
`digital_analog`	uint8	0-1	Point type
`range`	uint8	0-255	Unit selection
`decom`	uint8	0-2	Sensor status

Value Scaling:

- Temperature: value = actual × 10 (720 = 72.0°F)
- Percentage: value = actual × 10 (500 = 50.0%)

Output Structure

Str_out_point

Size: 46 bytes **Max Count:** 64

C++

```
typedef struct {
    char description[21];
    char label[9];
    uint8_t auto_manual;        // 0=Auto, 1=Manual
    unsigned short value;       // Output value
    unsigned char digital_analog;
    unsigned char range;
    unsigned char control;
    unsigned char decom;
    unsigned char low_voltage;
    unsigned short pwm_period;
    unsigned char sub_id;
    unsigned char sub_product;
    unsigned char unused[8];
} Str_out_point;
```

Variable Structure

Str_variable_point

Size: 46 bytes **Max Count:** 64

C++

```
typedef struct {
    char description[21];
    char label[9];
    uint8_t auto_manual;
    unsigned short value;
    unsigned char digital_analog;
    unsigned char range;
    unsigned char control;
    unsigned char unused[13];
} Str_variable_point;
```

Program Structure

Str_program_point

Size: 37 bytes **Location:** ud_str.h line 514 **Max Count:** 16

C++

```
typedef struct {
    union {
        char description[STR_PRG_DESCRIPTION_LENGTH+1];  // 21 bytes
    };
    union {
        char label[STR_PRG_LABEL_LENGTH+1];  // 9 bytes
    };
    unsigned char bytes[400];   // Program bytecode
    unsigned char on_off;       // 0=Off, 1=On
    unsigned char auto_manual;  // 0=Auto, 1=Manual
    unsigned char com_prg;      // Communication flag
    unsigned char errcode;      // Error code
    unsigned char unused[3];
} Str_program_point;
```

Field Details

Field	Type	Description
`description`	string	Program name
`label`	string	Short name
`bytes`	array	Compiled program code (400 bytes)
`on_off`	uint8	0=Off, 1=On
`auto_manual`	uint8	0=Auto, 1=Manual
`com_prg`	uint8	Communication status
`errcode`	uint8	Last error code

Controller Structure

Str_controller_point

Size: 24 bytes **Location:** ud_str.h line 543 **Max Count:** 16

C++

```
typedef struct {
    unsigned char input;        // Input point index
    short input_value;          // Input value
    short value;                // Output value
    unsigned char setpoint;     // Setpoint point index
    short setpoint_value;       // Setpoint value
    unsigned char units;        // Unit type
    unsigned char auto_manual;  // 0=Auto, 1=Manual
    unsigned char action;       // 0=Direct, 1=Reverse
    short proportional;         // P gain
    short reset;                // I gain (integral time)
    char bias;                  // Output bias
    unsigned char rate;         // D gain (derivative time)
} Str_controller_point;
```

PID Parameters

Field	Range	Description
`proportional`	0-100	Proportional gain
`reset`	0-100	Integral time
`rate`	0-100	Derivative time
`bias`	-100 to +100	Output offset
`action`	0-1	0=Direct acting, 1=Reverse acting

Monitor Structure

Str_monitor_point

Size: 133 bytes **Location:** ud_str.h line 639 **Max Count:** 12

C++

```
#define MAX_POINTS_IN_MONITOR 14

typedef struct {
    union {
        char label[STR_MONITOR_LABEL_LENGTH+1];  // 9 bytes
    };
    unsigned char inputs[MAX_POINTS_IN_MONITOR];   // Input point indices
    unsigned char range[MAX_POINTS_IN_MONITOR];    // Range for each input
    unsigned char second_interval_time;
    unsigned char minute_interval_time;
    unsigned char hour_interval_time;
    unsigned short max_time_length;   // Maximum log duration
    unsigned char num_inputs;         // Number of inputs configured
    unsigned char an_inputs;          // Number of analog inputs
    unsigned char status;             // 0=Stopped, 1=Running
    unsigned long next_sample_time;   // Next sample timestamp
    unsigned char unused[85];
} Str_monitor_point;
```

Trendlog Timing

Field	Range	Description
`second_interval_time`	0-59	Sample every N seconds
`minute_interval_time`	0-59	Sample every N minutes
`hour_interval_time`	0-23	Sample every N hours
`max_time_length`	0-65535	Max log duration (minutes)

Sample Intervals:

- Fast logging: 1-10 seconds
- Normal: 1-5 minutes
- Slow: 1-24 hours

Global Data Arrays

Structures are stored in vectors indexed by panel and point:

```cpp
// Defined in global_variable.h
vector<vector<Str_in_point>> g_Input_data;
vector<vector<Str_out_point>> g_Output_data;
vector<vector<Str_variable_point>> g_Variable_data;
vector<vector<Str_program_point>> g_Program_data;
vector<vector<Str_controller_point>> g_controller_data;
vector<vector<Str_monitor_point>> g_monitor_data;
```

Access pattern:

```cpp
// Read input 5 from panel 0
Str_in_point& input = g_Input_data[0].at(5);
int temp = input.value;  // Get value

// Write output 3 on panel 1
g_Output_data[1].at(3).value = 750;  // Set to 75%
```

Temporary Buffers

During reads, data first goes to temporary arrays:

```cpp
// Defined globally
Str_in_point s_Input_data[BAC_INPUT_ITEM_COUNT];
Str_out_point s_Output_data[BAC_OUTPUT_ITEM_COUNT];
Str_program_point s_Program_data[BAC_PROGRAM_ITEM_COUNT];
// etc.
```

Read flow:

```text
Device → BACnet → s_Program_data[] → g_Program_data[panel][]
```

Union Usage

Structures use unions for backward compatibility:

```C++
union {
    char description[STR_IN_DESCRIPTION_LENGTH+1];  // 21
};
```

This allows old code to access fields differently while maintaining same memory layout.

Packing and Alignment

Structures are packed to match device memory:

```C++
#pragma pack(push, 1)  // Byte-aligned packing
typedef struct { ... } Str_in_point;
#pragma pack(pop)
```

Important: No padding between fields - exact byte layout matters for network transmission.

See Also

- BACnet Commands - Reading/writing structures
- Message 17: GET_WEBVIEW_LIST - Using structures in messages
- Platform Overview - Architecture